

School of Computing Technologies

INTE2628/2679 Developing Blockchain Applications

Assignment 2

	Assessment Type: Group assignment, with a maximum of 2 students per group. Marks are awarded based on how closely your submission meets the stated requirements. Clarifications and updates will be provided via Canvas announcements and the relevant discussion forums.
	Deadline: Week 9, Friday, 11:59 pm Deadlines will not be brought forward. However, extensions may be granted — please check Canvas → Syllabus or Canvas → Assignments → Assignment 2 for the latest updates. A standard university late penalty of 10% per calendar day applies, unless an approved extension or special consideration applies.
	Weighting: 40 marks (Contributes 40% to the final course grade)

1. Overview

The goal of this assignment is to design and implement an interactive, multi-phase decision voting platform on the Ethereum blockchain. You will develop a smart contract that manages voting sessions, enforces rule-based participation, and produces transparent decision outcomes. Your system must also include a frontend interface that allows users to interact with the deployed smart contract through MetaMask and Web3.js.

Your system must support:

- Multi-phase voting workflow
- Role-based access control
- Strict voting and participation rules
- Frontend feedback based on contract interaction and system state
- Frontend integration with MetaMask and Web3.js
- Lifecycle management for multiple voting rounds

You are required to submit a working smart contract, frontend source files, and a detailed documentation file that includes:

- A clear explanation of your smart contract and frontend design
- Screenshots showing contract deployment, MetaMask interaction, and frontend integration

- Test evidence demonstrating correct logic, rule enforcement, and role-based behaviour

Note: If you have any questions, use the relevant Canvas discussion forums or ask during Practical sessions. Please phrase queries in a clear and general manner.

Submission Guidelines:

Submit your solution on Canvas as two separate files:

1. a **ZIP** folder containing the required **source code files**; and
2. a **single PDF documentation file**.

Do not place the PDF inside the ZIP archive. Upload the ZIP folder and PDF file separately.

2.Assessment Criteria

This assessment will determine your ability to:

- Follow the requirements provided in this document and in the lessons.
- Independently solve a problem by using concepts taught over the first eight weeks of the course.
- Submit work by the required deadline and follow the specified submission requirements.

3.Learning Outcomes

- This assessment supports CLOs 1, 2, 3 and 4 for INTE2628, and the corresponding postgraduate CLOs for INTE2679. Please refer to the official course guide for the CLO wording applicable to your enrolled course code.

4.Assessment details

Please ensure that you have read **Sections 1 to 3** of this document before going further. Assessment details are provided on the **next page**.

INTE2628/2679 Developing Blockchain Applications

Assignment 2 – Decision Voting Platform with Interactive Workflow Group (max 2 students)

Scenario & Objective

A community group wants to use blockchain technology to support repeated decision-making activities, such as choosing an event option, selecting a group proposal, or deciding how shared resources should be allocated. Unlike a one-off vote, this process needs to support multiple voting sessions over time, with different topics, options, and participants.

In Assignment 1, you developed a smart contract for managing a rule-based voting process with basic lifecycle control. In this assignment, you will extend this idea into an interactive decision voting platform that supports multiple voting sessions, frontend interaction, and more structured role- and phase-based workflows.

You may reuse concepts from Assignment 1, but the submitted smart contract must be adapted to meet the Assignment 2 requirements, including text-based options, repeated voting sessions, participant eligibility management, result reveal, and frontend interaction.

Your solution must also include a browser-based interface so that users can interact with the deployed smart contract through MetaMask and Web3.js. The interface should help each user understand their current role, the current voting status, and the actions they are allowed or not allowed to perform.

The objective is to build a reusable blockchain-based voting platform that combines smart contract logic with a clear interactive workflow to support transparent and rule-based decision-making.

Platform Expectations

Your solution should behave as an interactive voting platform where:

- The Admin manages voting round setup, eligibility, voting closure, and result reveal.
- Participants can connect their wallet, view voting information, cast votes, and view results only when the system rules allow.
- Each voting round is handled independently, with data from previous rounds properly separated or cleared.
- Access control and voting rules are enforced by the smart contract rather than by manual supervision.
- The interface clearly communicates meaningful system states, such as whether voting is open, whether a user is eligible to vote, and whether results are available.

Required Capabilities

Your platform must satisfy the following functional requirements.

1. Initialising the Platform

The platform must allow the voting system to be initialised when the smart contract is deployed. The account that deploys the smart contract must automatically become the Admin. The Admin address must not be manually entered or hardcoded in the source code.

The Admin is responsible for managing the voting lifecycle, including preparing voting rounds, managing voter eligibility, ending voting, and revealing results where required.

2. Preparing or Resetting a Voting Round

After the platform is initialised, the Admin must be able to prepare a voting round. Each voting round must include:

- a custom voting topic; and
- a set of text-based voting options.

The voting round must also satisfy the following rules:

- the voting topic must not be empty;
- the voting round must contain at least 2 voting options; and
- each option must contain non-empty text.

The platform must support repeated voting rounds over time. A new voting round must not replace an active voting round. Once a voting round has been prepared and voting is open, it must be ended before the system is reset or prepared for the next round.

Before a new round begins, all round-specific data from the previous round must be cleared, including previous options, vote records, voter participation records, result information, and voting ineligibility records.

3. Casting Votes

Participants must be able to view the available voting options and select one option to vote for.

The voting rules must ensure that:

- each eligible participant can cast only one vote per round;
- duplicate votes are prevented;

- votes for invalid or unavailable options are rejected; and
- the Admin is not permitted to vote.

Unless excluded by the Admin, any connected non-Admin wallet address may be treated as a participant for the current voting round. A separate participant registration system is not required for this assignment.

4. Voter Eligibility Management

Some participants may be ineligible to vote in a specific round, for example because of a conflict of interest or a rule violation.

Only the Admin may manage voting eligibility. The system must allow the Admin to:

- exclude a participant from voting in the current round;
- reinstate a participant's voting rights for the current round; and
- view the current list of excluded participants.

Voting ineligibility is round-specific. All exclusion records from a previous round must be cleared before the next voting round begins.

5. Ending a Voting Round

The Admin must be able to end the current voting round.

Once a voting round has ended:

- no further votes may be submitted;
- the recorded votes for that round must remain final; and
- the system must move to a state where results can be revealed or viewed according to the required rules.

6. Revealing and Viewing Results

Ending a voting round and revealing results are separate lifecycle steps. Vote counts, winners, and result summaries must not be available while voting is still active. Interim vote counts or live results must not be revealed during the voting phase.

Only after the voting round has ended can the Admin reveal the results. Once results are revealed:

- the winning option or options with the highest number of votes must be displayed;
 - If multiple options have the same highest number of votes, all tied options must be identified as winners.
- users must be able to view a clear summary of the results; and

- the result summary must show each option together with the number of votes it received.

Each participant must only be able to view their own vote and their own voting status. Participants must not be able to view another participant's vote or voting status.

Where required for managing the voting process, the Admin may view participant voting status, such as whether a participant is eligible or has voted. However, individual vote choices must not be exposed to other participants. Result summaries should show option-level vote counts, not a public list of who voted for which option.

7. Public Session Information and Connected Wallet Information

The platform must allow users to view general voting session information, including:

- the current voting topic;
- the available voting options; and
- the current voting status.

The interface must also allow each connected user to view their own wallet-related information, including:

- wallet address;
- connected Ethereum network; and
- current balance.

Interface Requirements

Your platform must provide a browser-based web interface that allows users to interact with the deployed smart contract through MetaMask and Web3.js. The interface must respond to the connected wallet address, the user's role, and the current voting state.

Visual design is not assessed. However, the interface must be clear, logically organised, and provide meaningful feedback to users.

General Requirements

The interface must:

- be accessible through a browser, such as through localhost or a hosted page;
- connect to the smart contract deployed through Remix;
- allow users to connect their MetaMask wallet;
- display relevant voting session information clearly; and
- provide prompts, warnings, or visual cues when user actions are restricted or when the system state changes.

The interface must clearly indicate:

- whether a voting round has been prepared;
- whether voting is currently open or ended;
- whether the connected user is the Admin or a participant;
- whether the connected participant is eligible to vote;
- whether the connected participant has already voted;
- whether results are available to view; and
- whether a selected action is currently permitted or restricted.

Dynamic Interface Logic

No separate username/password login system is required. User role and access should be determined based on the connected wallet address and contract state.

Your system must provide one interface page that includes both Admin and participant functionalities. Users should access the platform by connecting their wallet. The interface must adapt based on:

- the connected wallet address;
- the user role: Admin or participant;
- the current voting phase, such as setup, voting, or reveal/results stage; and
- the connected participant's voting status, such as eligible, ineligible, not voted, or already voted.

Although all functionalities are included on one page, action elements such as buttons and input fields must remain visible and available for testing during the demo. Do not hide or disable restricted action elements. Restricted actions must remain visible and testable during the demo, so that the smart contract's rule enforcement can be evaluated.

Instead, when a connected user is not permitted to perform an action, the interface must show a clear warning message next to the relevant action element. The warning message should explain why the action is currently not permitted, based on the user role, current voting phase, or voting status.

For example:

- A participant should see a warning near the "Submit Vote" button if voting has ended, if they are ineligible, or if they have already voted.
- The Admin should see a warning near the "Reveal Results" button if voting is still active.
- A non-Admin user should see a warning near Admin-only actions.

The smart contract must still enforce the required rules and return appropriate error messages when invalid actions are attempted. Interface warnings alone are not sufficient.

Note: UI appearance is not graded. However, interface logic, role-based behaviour, state-based feedback, and smart contract interaction will be assessed during the live in-class demo.

Submission Requirements

You must submit the following two components on Canvas:

1. **Source code files**, submitted as one ZIP folder
2. **System documentation**, submitted as one PDF file

This is a group assignment. **Only one group member needs to submit the assignment on behalf of the group. Other group members do NOT need to submit the same files separately.** However, the documentation file must clearly list the full name and student ID of every group member.

Do not place the PDF documentation file inside the ZIP folder. Upload the ZIP folder and the PDF file separately.

1. Source Code Files

Your ZIP folder must include all source code files required to run and demonstrate your platform, including:

- a Solidity smart contract implementing the voting platform logic;
- HTML and JavaScript files for the browser-based interface;
- any optional CSS files used for layout or clarity; and
- inline comments explaining the purpose of major code sections and key functions.

Your submitted source files must be complete and runnable after being downloaded from Canvas. During the live demo, you may be required to re-download and run the submitted files.

2. System Documentation File

The documentation file must be submitted as a single PDF.

Your documentation must include at least the following information:

i. Group information

- Full name and student ID of each group member.
- A statement identifying the group member who submitted the assignment on Canvas.

ii. System overview

- A short summary of the goal of your platform and how it behaves.
- A visual diagram showing the overall system flow, including the smart contract, frontend interface, MetaMask/Web3.js interaction, and user roles.
- A brief explanation of the voting lifecycle supported by your system.

iii. Functional breakdown

- A clear explanation of the main smart contract functions and how they relate to the required platform behaviours.
- A summary of what the Admin can do.
- A summary of what participants can do.
- A summary of how the interface responds to user roles, voting status, and system state.

iv. Screenshots with captions

You must include screenshots with clear captions showing at least the following:

- smart contract deployment or redeployment through Remix;
- voting round setup, including topic and option input;
- a participant casting a vote through the interface;
- role-based or state-based warning messages shown in the interface;
- final result display after results are revealed; and
- relevant MetaMask or transaction interaction evidence where appropriate.

v. Reflection

A short reflection on design challenges, limitations, or possible improvements.

Additional Notes

- This is a group assignment, with a maximum of 2 students per group. Only one group member needs to submit the assignment on Canvas on behalf of the group. Other group members do not need to submit the same files separately. The PDF documentation must clearly list the full name and student ID of every group member.
- All submitted code must be original and must be based on concepts, tools, and development approaches covered in Weeks 1–8 of this course, unless otherwise approved by the teaching team.
- External libraries, code generators, pre-built frameworks, or unauthorised templates are not allowed. Web3.js is permitted for connecting the browser-based interface to the deployed smart contract.
- Visual appearance is not assessed. Marks are awarded based on smart contract logic, rule enforcement, user flow, interface clarity, frontend–contract interaction, documentation, and the live demonstration.
- Each group must attend the scheduled live in-class demonstration of their system. All group members are expected to attend and be able to explain the submitted work. If a group does not attend the scheduled demo without an approved arrangement, the teaching team may be unable to assess the system, and marks for demonstration-dependent components may not be awarded.
- During the live demo, students must use the source files submitted on Canvas. Students may be required to re-download the submitted ZIP file, extract it, redeploy the smart contract through Remix, and run the frontend interface. Please ensure that your submitted files are complete and can run correctly after being downloaded from Canvas.
- No substantial debugging, code modification, or replacement of submitted files will be permitted during the live demo. Basic setup actions, such as opening files, connecting MetaMask, redeploying the submitted contract, or running the submitted frontend, are allowed.
- Students are responsible for ensuring that they have sufficient Sepolia ETH and a working MetaMask setup for the live demo. If required functionalities cannot be demonstrated because of insufficient Sepolia ETH, wallet setup issues, or missing submitted files, this will be considered the responsibility of the group.

Rubric/assessment criteria for marking

Assessment Evidence and Live Demonstration

A working program alone does not automatically guarantee full marks. This assessment is marked based on the submitted source code, documentation, live demonstration, and the group's ability to explain and justify the submitted work.

During the live in-class demonstration, the teaching team may ask any group member to demonstrate required functions, explain design choices, interpret outputs or error messages, and answer questions about the smart contract, frontend interaction, rule enforcement, and documentation.

If a required feature cannot be demonstrated or satisfactorily explained during the live demo, it may be treated as not satisfactorily evidenced for marking purposes, even if it appears in the submitted code or report.

Criteria	Ratings					Total
Decision Voting Platform: Smart Contract, Frontend, Documentation, and Live Demonstration	The submitted platform meets the assignment requirements to a high standard. The smart contract correctly implements the required voting workflow, including Admin assignment, voting round setup, text-based options, repeated rounds, reset of round-specific data, voting eligibility, one vote per eligible participant, prevention of duplicate or invalid votes, Admin voting restriction, exclusion	The submitted platform meets most assignment requirements. The main smart contract workflow, frontend interaction, documentation, and live demonstration are mostly complete and functional. Minor issues may exist in edge cases, reset behaviour, eligibility handling, result reveal, privacy protection, interface warnings, wallet/session display, documentation	The submitted platform implements the main functionality but contains noticeable flaws in important areas. The smart contract may run but may not fully enforce required rules such as eligibility, reset, phase control, invalid-action handling, result reveal, or privacy. The frontend may only partially support required role-based or state-based behaviour. Documentation	Basic voting functionality is attempted, but multiple required components are missing, broken, or poorly integrated. The system may allow simple voting but does not reliably support the required multi-round workflow, access control, eligibility management, reset, result reveal, interface feedback, or privacy requirements. The frontend or documentation may	The submission is incomplete or does not demonstrate a working decision voting platform. Major components are missing or faulty, the smart contract may not compile or deploy, the frontend may be missing or unable to connect, required documentation may be largely absent, or the submitted files cannot be meaningfully	40 pts

	and reinstatement of participants, ending voting, result reveal, option-level result summary, and protection of individual vote privacy. The frontend connects to the deployed smart contract through MetaMask and Web3.js, provides clear one-page Admin and participant interaction, displays relevant session and wallet information, and gives clear role-based and state-based warnings while keeping restricted actions visible and testable. The documentation is	detail, or explanation during the demo. However, the core platform works, the main rules are mostly enforced, and group members can explain most parts of the submitted work. 28 - 31 pts	may be incomplete or unclear in several places. During the demo, the group can show some required workflow components but may have difficulty demonstrating or explaining important features, design choices, outputs, or frontend–contract interactions. 24 - 27 pts	be limited. During the demo, key functionality may be difficult to demonstrate, and group members may show limited understanding of major parts of the submitted work. 20 - 23 pts	demonstrated. A program that appears to run but cannot be satisfactorily demonstrated or explained may also fall within this range, depending on the extent of the missing evidence and understanding. 0 - 19 pts	
--	---	--	--	---	--	--

complete, well organised, and includes the required group information, system overview, diagram, functional breakdown, screenshots, and reflection. The submitted files can be used during the live demo, and group members can clearly demonstrate, explain, and justify the submitted system, including the smart contract logic, frontend interaction, rule enforcement, outputs, and error messages.

32 - 40 pts